

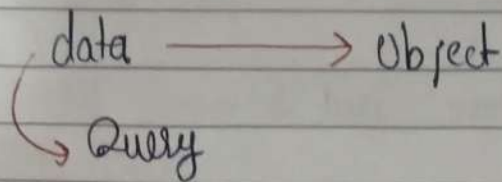
Collection Framework

Page No.:

Date:

YOUVA
STELLAR

support of Data Structures



arr = [3, 7, 10, 1, 4, 12], 24, 0
constant time $\longrightarrow O(1)$

Query: largest no. : 24

$O(N)$

arr (sort)

[0, 1, 3, 4, 7, 10, 12, 24]

Query: $\longrightarrow O(1)$

return arr[arr.length - 1]

~~insert~~ Insertion: $O(N)$

Arrays

```
class LinkedList {  
    }  
}
```

normal array: int[] arr = new int[10];

arr[10] = index out of bound

Dynamic Array

↳ no fixed size

```
class DynamicArray {
    private int[] arr;
    DynamicArray (int size) {
        arr = new int [size];
    }
    add (int el) { }
    get (int index) { }
    insert (int index, int elem) { }
}
```

4	10	13	15	17
0	1	2	3	4

add()

4	10	13	15	17	21				
0	1	2	3	4	5	6	7	8	9

add(4);
add(10);
add(13);
add(15);
add(17);
add(21);

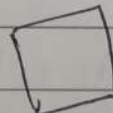
insert(1, 25)

⇒ ArrayList (given by java)

Problem: Insertion

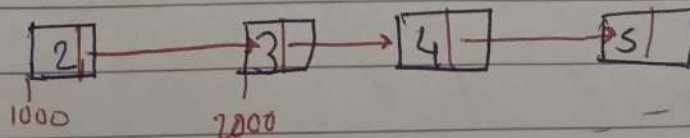
↳ LinkedList

2	3	4	5
0	1	2	3



```
class Node {
    int data;
    Node ref;
}
```

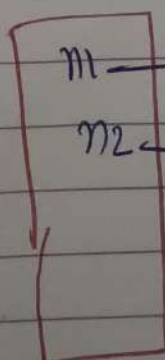
Node :



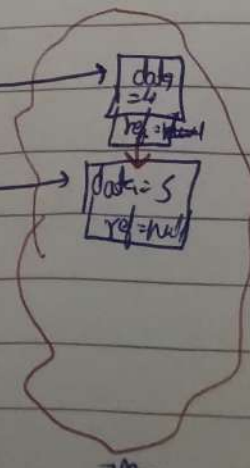
Node n1 = new Node();
n1.data = 4;

Node n2 = new Node();
n2.data = 5;

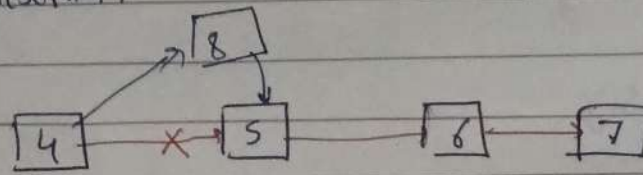
n1.ref = n2;



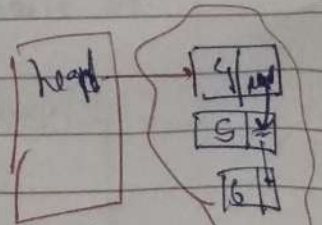
stack



Insertion



add(7)



Stack

Heap

```

class Linked List {
    private static class Node {
        int data;
        Node next;
    }
    Node head;
    public void add (int el) {
    }
}
  
```

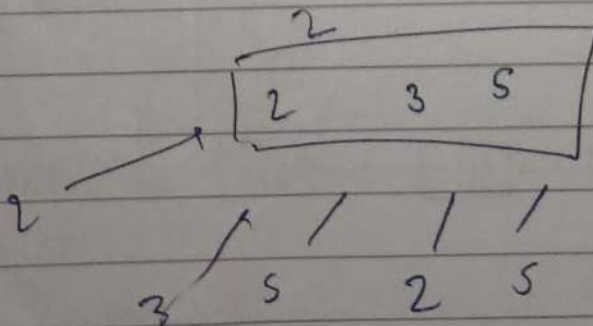
```

add (4);
add (5);
add (6);
add (7);
insert (8, 1);
  
```

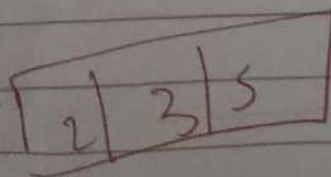
Linked List

Set:

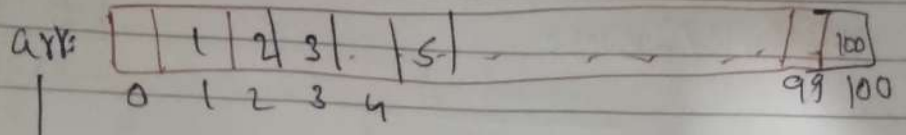
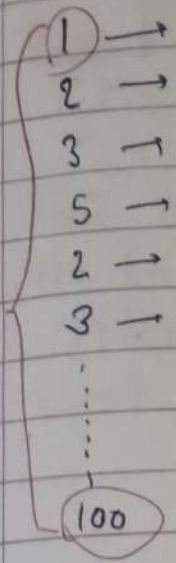
└ Duplicates are not allowed



2, 3, 5



Constant
time
Query

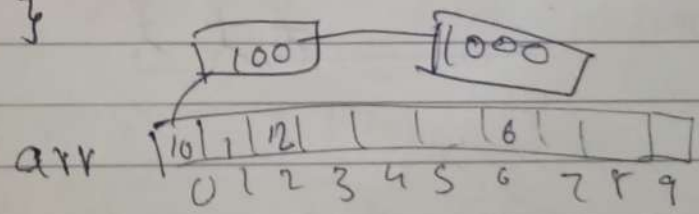
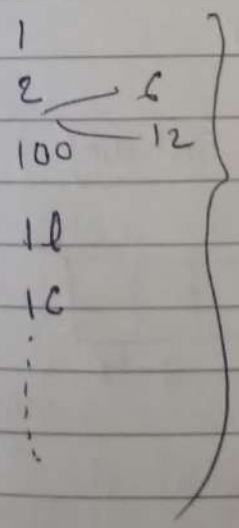


- ① Duplicates not allowed
- ② constant time →
element → exist / not exist

```
add (int el) {  
    arr[el] = el;  
}
```

0 (1)

```
contains (int el) {  
    return arr[el] == el;  
}
```



arr.length → 10

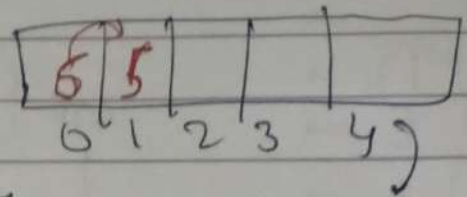
$(1 \% \text{arr.length}) \rightarrow [0 - 9]$

```
(chaining) contains (int el) {  
    arr[el % arr.length] == el;  
}
```

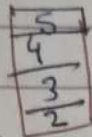
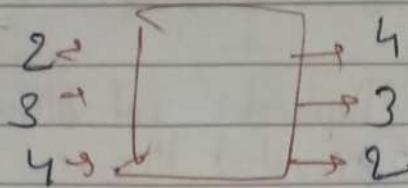
hashes for string

HashSet
Linked Hash set } java

* Stack/Queues:



2, 3, 4



5, 3, 3, 2

②

③

Stack

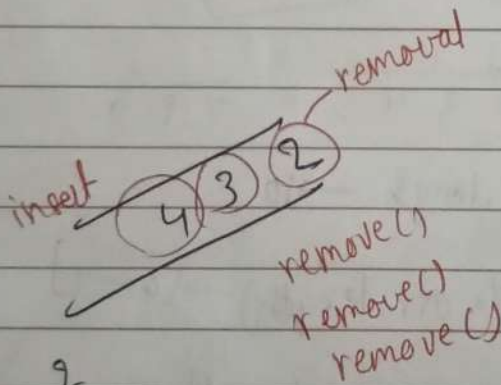
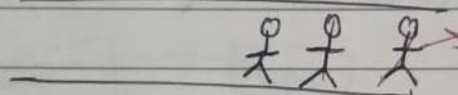
LIFO

④

Last In first out

```
class Stack {
    int arr;
    add(s);
    remove();
}
```

Queue:



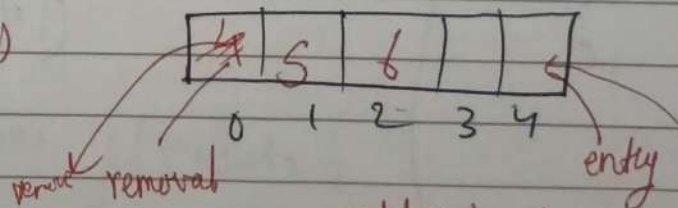
2

3

4

FIFO

First In First out



add(4)

add(5)

add(6)

remove()

java: Queue
Array Deque
Stack

* Maps

Key → value

→ Aditya → 101
→ Rohit → 102
→ Rohan → 103

$O(1)$

① Aditya (101) $O(1)$

② No Duplicate keys (X)

Aditya ~~→ 105~~

set → 1, 2, 3

→ (key, value)

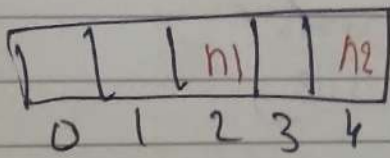
```
class Node {
    String key;
    int value;
    constructor()
```

}

```
add (name, rollNo) {
    Node n1 = new Node(' ', ' ');
    (hashCode(name) % 5) array.length
```

}

```
get (Aditya) {
    (101)
}
```



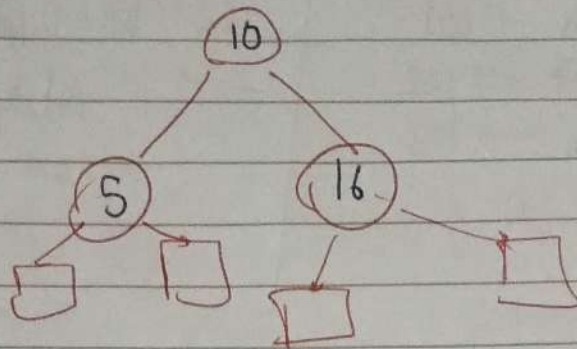
HashMap → Java
LinkedHashMap

```
get (key) {
    (hashCode (key) % 5)
}
```

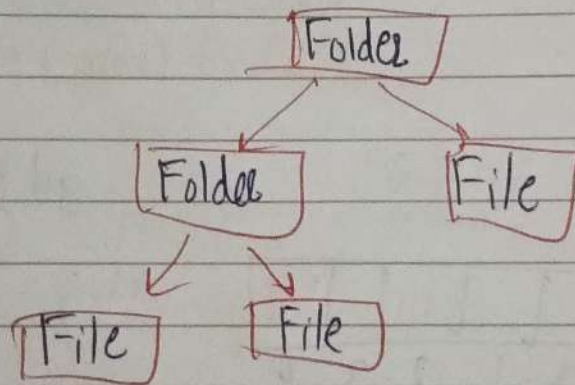

* Tree:

BST: Binary Search Tree

class TreeNode {
int data;
TreeNode left;
TreeNode right;



→ file & folder

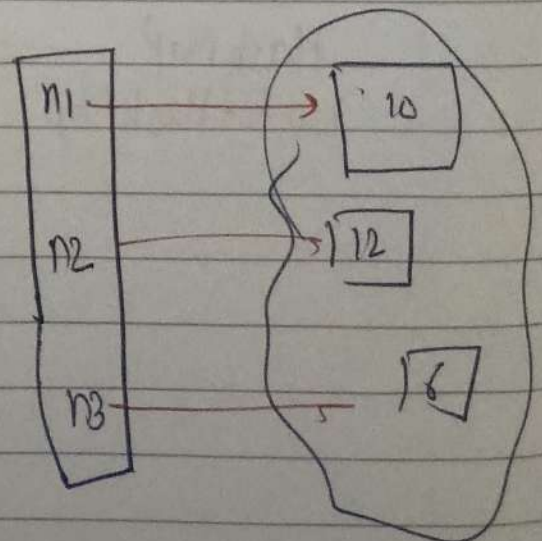
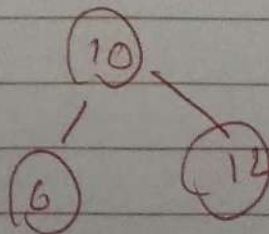


TreeNode n1 = new TreeNode();

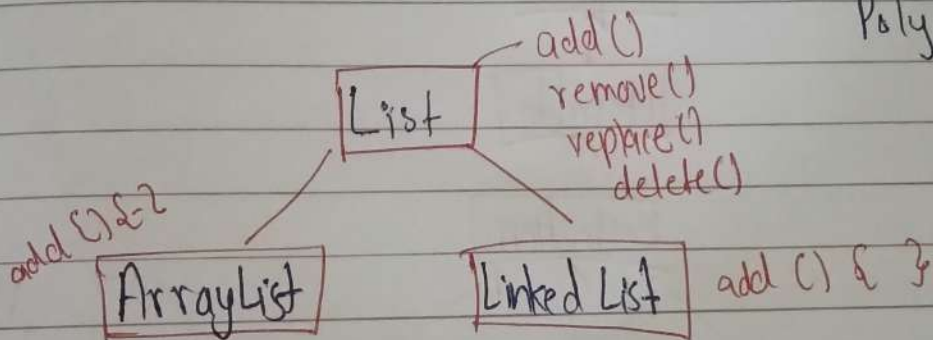
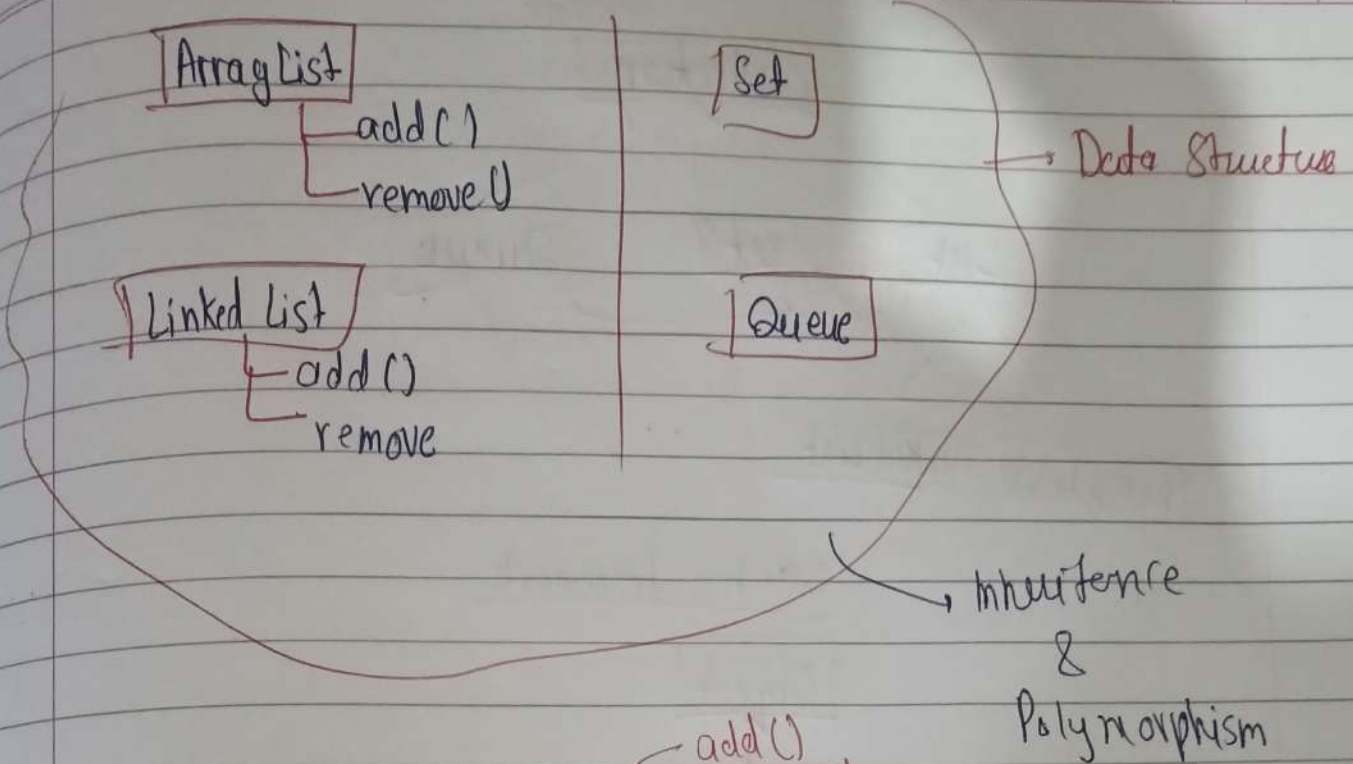
n1.data = 10;

n1.left = n3;

n1.right = n2;



Java: TreeSet
TreeMap



[2 | 3 | 4 | 5]

[2] → [3] → [4] → [5]

```
List<Integer> l = new ArrayList<>();
               new LinkedList<>();
```

```
l.add(4);
```